

ALUMNI NETWORK AND ENGAGEMENT PLATFORM

Database Management Systems Project
**DA2 - Database Implementation using SQL and PL/SQL
with Report Submission & Project Demonstration**

Team Members:

Sagarika Kaistha - 25BCE5091
Praveen G - 25BCE5092
Daksh Agarwal - 25BCE5098

TABLE OF CONTENTS

1. Introduction
2. Implementation Environment
3. Schema Implementation (SQL DDL)
4. PL/SQL Implementation
5. Views
6. Verification and Testing
7. Demonstration Queries
8. Conclusion

Appendix A - How to Run

Appendix B - Script Inventory

1. Introduction

In the previous deliverable (DA1), the Alumni Network and Engagement Platform was designed using the ER/EER model, normalized up to BCNF/4NF, and mapped to a relational schema of 21 relations. This report documents the implementation of that design as a working Oracle database.

- **22 tables** implementing every entity, supertype/subtype hierarchy, lookup table and junction table from the design, with all PK / FK / UNIQUE / CHECK constraints enforced by the DBMS.
- **18 sequences and 17 auto-increment triggers** providing Oracle-style surrogate key generation.
- **5 business-rule triggers** enforcing integrity that table constraints cannot express (capacity limits, date logic, audit trail).
- **4 stored procedures, 3 functions and 1 package** encapsulating event registration, donation recording with receipt generation, job applications and mentorship lifecycle.
- **3 views** defined in the DA1 design document.
- A **demonstration suite of 14 query scenarios**: joins, aggregates, subqueries, analytical functions, hierarchical queries, transaction control.

2. Implementation Environment

Component	Details
DBMS	Oracle AI Database 26ai Free (23.26.2.0.0), pluggable database FREEPDB1
SQL client	SQL*Plus Release 23.26.1.0.0 (Oracle Instant Client)
Hosting	Docker container gvenzl/oracle-free on macOS; port 1521
Project schema	User ALUMNI with CREATE TABLE / VIEW / PROCEDURE / TRIGGER / SEQUENCE privileges
Scripts	sql/00_drop_all.sql ... sql/run_all.sql executed in order via SQL*Plus @

Every script is plain Oracle SQL/PL-SQL, so the same files run unchanged on Oracle Live SQL or any college lab Oracle 11g+ installation.

3. Schema Implementation (SQL DDL)

3.1 Data Type Mapping

Design type	Oracle type	Notes
INT / BIGINT	NUMBER(p)	precision per attribute
DECIMAL(m,n)	NUMBER(m,n)	e.g. DONATION.Amount NUMBER(12,2)
VARCHAR(n)	VARCHAR2(n)	Oracle-recommended variable text
TEXT	CLOB	descriptions, posts, comments, feedback
BOOLEAN	NUMBER(1) + CHECK IN (0,1)	IsActive, IsPinned, IsAnonymous
DATETIME DEFAULT NOW()	TIMESTAMP DEFAULT SYSTIMESTAMP	CreatedAt, PostedDate, ...
AUTO_INCREMENT	SEQUENCE + BEFORE INSERT trigger	see section 4.1
TIME	VARCHAR2(5) + REGEXP CHECK 'HH24:MI'	EVENT.EventTime

3.2 Tables Implemented and Sample Data Volume

Table(s)	Role in design	Rows
PERSON	Supertype (EER generalization)	15
ALUMNI / STUDENT / ADMIN	Subtypes, PK = FK to PERSON	10 / 3 / 2
DEPARTMENT / BATCH	Academic structure	4 / 5
COMPANY / COMPANY_SIZE	Employers + size lookup	8 / 5
SKILL / ALUMNI_SKILL	Skill catalogue + M:N junction	8 / 14
EVENT_TYPE / EVENT	Event lookup + events	4 / 5
EVENT_REGISTRATION	M:N alumni-event with attendance status	11
DONATION	Contributions; auto receipt numbers	12
JOB / JOB_APPLICATION	Job portal + M:N applications	6 / 8
MENTORSHIP_AREA / MENTORSHIP	Domains + engagements	5 / 4
FORUM / POST / COMMENT_TABLE	Discussion hierarchy, self-referencing replies	3 / 6 / 8
ALUMNI_AUDIT	Trigger-populated audit trail	10

3.3 Constraint Enforcement (verified via USER_CONSTRAINTS)

Type	Count	Examples
PRIMARY KEY	22	one per table; composite PKs on the three junction tables
FOREIGN KEY	30	subtype FKs ON DELETE CASCADE; HOD/company SET NULL; self-referencing comment parent
UNIQUE	12	Email, StudentID, ReceiptNumber, TransactionID, (Mentor,Mentee,Start), (Job,Applicant)
CHECK	65	gender, CGPA range, rating 1-5, payment methods, attendance status, HH24:MI time format

All 18 indexes recommended by the design document were created for frequent join/filter paths.

4. PL/SQL Implementation

4.1 Sequences and Auto-Increment Triggers

```
CREATE SEQUENCE seq_person START WITH 1 INCREMENT BY 1 NOCACHE;

CREATE OR REPLACE TRIGGER trg_person_bi BEFORE INSERT ON PERSON
FOR EACH ROW
BEGIN
    :NEW.PersonID := NVL(:NEW.PersonID, seq_person.NEXTVAL);
END;
/
```

Inventory: 18 sequences (including seq_receipt for receipt numbers) and 17 auto-increment triggers.

4.2 Business-Rule Triggers

#	Rule enforced	Trigger	Error	Verified result
R1	Donation cannot be future-dated; missing receipts auto-generated RCP-1-	TRG_DONATION _BUSINESS	ORA-20001	rejected; receipt generated
R2	Registration rejected when event full or already conducted	TRG_EVENTREG _BUSINESS	ORA-20002/ -20005	'FULL (3/3)' observed
R3	Mentor <> mentee; end>=start; setting end date completes engagement	TRG_MENTORSH IP_BUSINESS	ORA-20003/ -20006	self-mentoring blocked
R4	JOB.ExpiryDate later than posting date	TRG_JOB_BUSIN ESS	ORA-20004	rejected
R5	ALUMNI insert/update/delete logged to ALUMNI_AUDIT	TRG_ALUMNI_AU DIT (AFTER)	-	10 rows captured

```
CREATE OR REPLACE TRIGGER trg_eventreg_business
BEFORE INSERT OR UPDATE ON EVENT_REGISTRATION
FOR EACH ROW
DECLARE
    v_capacity EVENT.MaxCapacity%TYPE;
    v_booked   NUMBER;
    v_edate    EVENT.EventDate%TYPE;
BEGIN
    SELECT NVL(MaxCapacity, 999999), EventDate
    INTO v_capacity, v_edate
    FROM EVENT WHERE EventID = :NEW.EventID;

    IF TRUNC(SYSDATE) > v_edate THEN
        RAISE_APPLICATION_ERROR(-20005,
            'Cannot register: event already conducted.');
```

4.3 Stored Procedures

Procedure	Purpose	Key behaviours
sp_register_event(alumni, event)	Register for an event	duplicate-safe; capacity rule inherited from trigger

Procedure	Purpose	Key behaviours
sp_record_donation(..., OUT receipt)	Record a contribution	validates donor; generates receipt via RETURNING INTO
sp_apply_to_job(job, applicant, resume)	Apply to a posting	rejects closed/expired jobs and duplicates (ORA-20203..20206)
sp_close_mentorship(id, end, fb, rating)	Complete a mentorship	FOR UPDATE lock; only ACTIVE closable (ORA-20207)

4.4 Functions

Function	Returns
fn_total_donations(person_id)	Lifetime donation total of an alumnus
fn_event_occupancy(event_id)	Occupancy percentage (NULL when unlimited capacity)
fn_active_mentees(mentor_id)	Number of active mentees guided by an alumnus

4.5 Package PKG_ALUMNI_REPORTS

- top_donors(n) - RANK() over aggregated donations printed via DBMS_OUTPUT
- dept_alumni_summary - department-wise counts and average graduation year
- get_alumni_skills(person_id) - comma-separated skill list via cursor loop

5. Views

```
CREATE OR REPLACE VIEW AlumniProfileView AS
SELECT a.PersonID, p.FirstName, p.LastName, p.Email, a.GraduationYear,
       d.DeptName, c.CompanyName, a.CurrentPosition
FROM ALUMNI a
JOIN PERSON p      ON a.PersonID = p.PersonID
JOIN DEPARTMENT d  ON a.DeptID   = d.DeptID
LEFT JOIN COMPANY c ON a.CurrentCompanyID = c.CompanyID
WHERE a.IsActive = 1;
```

- EventSummaryView - registrations per event with organiser name and capacity
- DonationSummaryView - count, sum and last donation date per alumnus

6. Verification and Testing

6.1 Build verification

Executing 00 -> 01 -> 02 -> 03 on a clean schema produced zero ORA errors: 22 tables, 61 indexes, 18 sequences, 22 triggers, 3 views, 4 procedures, 3 functions and 1 package compiled VALID (USER_OBJECTS / USER_ERRORS).

6.2 Business-rule test results (spool log run_04_demo_queries.log)

```
CAUGHT: ORA-20001: Donation date cannot be in the future: 31-AUG-2026
CAUGHT: ORA-20002: Event is FULL (3/3 seats taken). Registration rejected.
CAUGHT: ORA-20003: Invalid mentorship: MentorID and MenteeID are the same person.
CAUGHT: ORA-20004: Job expiry date must be after the posting date.
CAUGHT: ORA-20205: You have already applied to "Senior Backend Developer".

[OK] Donation of Rs.5000.00 recorded from Ananya Iyer. Receipt: RCP-1012-2026
[OK] Alumnus #6 registered for "Annual Tech Meetup" (#5)
[OK] Application submitted for "Product Management Intern" (#2)
[OK] Mentorship #2 marked Completed (rating: 5/5).
```

6.3 Package output sample

```
---- TOP 5 DONORS ----
RANK  ID    NAME                AMOUNT
1      1    Rahul Sharma        75000.00
```

1	2	Priya Menon	75000.00
3	10	Meera Krishnan	60000.00
4	3	Arjun Patel	55000.00
5	9	Rohit Verma	35000.00

6.4 Audit trail evidence

AUDIT_ID	OPERATION	NEW_VALUES
1	INSERT	Year=2019, Dept=1, Pos=Senior Software Engineer
...		
10	INSERT	Year=2022, Dept=4, Pos=Founder - AgriTech Startup

(Screenshot placeholders: insert live screenshots of the spool logs during the viva demo.)

7. Demonstration Queries

Q#	SQL concept showcased
Q1-Q2	Inner / LEFT OUTER joins across 3-4 tables; LISTAGG aggregation
Q3-Q4	GROUP BY + HAVING; subquery comparing against average
Q5	Correlated NOT EXISTS (alumni who never donated)
Q6	Analytical RANK() over event popularity
Q7	Hierarchical CONNECT BY PRIOR over the self-referencing comment tree
Q8	The three design views
Q9	Transaction control: SAVEPOINT / ROLLBACK TO SAVEPOINT with a CHECK violation
Q10-Q11	Functions and package invocation from SQL and SQL*Plus EXEC
Q12-Q13	Procedures: happy paths plus each business-rule rejection
Q14	Audit trail populated by trigger R5

8. Conclusion

The complete DA1 design is now a running Oracle database. Every entity, relationship and constraint from the ER/EER model is enforced physically; PL/SQL adds the procedural layer (key generation, business validation, auditing, reporting API). The build is fully scripted and reproducible, and the demonstration suite shows both successful flows and deliberate violations being caught by the database itself.

Future work: web/application front-end, partitioning DONATION by year, fine-grained roles, REST endpoints over the package API.

Appendix A - How to Run

```
# Local SQL*Plus
cd <project-root>
sqlplus system/<password>@localhost:1521/FREEPDB1
  CREATE USER alumni IDENTIFIED BY alumni123 QUOTA UNLIMITED ON USERS;
  GRANT CREATE SESSION, CREATE TABLE, CREATE VIEW, CREATE PROCEDURE,
    CREATE TRIGGER, CREATE SEQUENCE TO alumni;

CONNECT alumni/alumni123@localhost:1521/FREEPDB1
@sql/01_schema.sql
@sql/02_plsql.sql
@sql/03_seed_data.sql
SET SERVEROUTPUT ON
@sql/04_demo_queries.sql

# Oracle Live SQL: paste 01 -> 02 -> 03 -> 04 into the worksheet,
# enable DBMS_OUTPUT in settings.
```

Appendix B - Script Inventory

File	Contents
sql/00_drop_all.sql	Tears down views, PL/SQL objects, triggers, tables, sequences
sql/01_schema.sql	22 tables, constraints, 18 indexes, audit table
sql/02_plsql.sql	Sequences, auto-increment + business triggers, views, procedures, functions, package
sql/03_seed_data.sql	Sample data + row-count sanity check
sql/04_demo_queries.sql	14 demonstration scenarios
sql/run_all.sql	Master runner producing logs/run_*.log spools
logs/	Captured execution logs used as report evidence